

# Projeto Coin - Plano de Implementação Detalhado

Este documento especifica a arquitetura de software, design system e fluxo de trabalho para o desenvolvimento da página da Semana da Inteligência Artificial (Coin) da HealthTech Júnior Einstein.

## 1. Design System e Identidade Visual

### 1.1. Paleta de Cores Oficial

O design exige um perfil tecnológico com contraste entre tons neon (azul/roxo) e amarelo para os destaques. As cores oficiais devem ser configuradas como variáveis CSS para facilitar a reutilização e garantir a consistência com o tema "O FUTURO DA SAÚDE É COOPERATIVO E INTELIGENTE".

| Nome da Variável CSS    | Código Hexadecimal | Uso Principal                                          |
|-------------------------|--------------------|--------------------------------------------------------|
| --coin-neon-purple      | #4D28FF            | Destaques neon, hover states, efeitos de brilho        |
| --coin-neon-blue        | #4151FF            | Tons fluidos e dinâmicos para background               |
| --coin-dark-blue        | #1D27B4            | Fundos primários e contraste profundo                  |
| --coin-base-blue        | #2430DB            | Elementos secundários e gradientes                     |
| --coin-yellow-highlight | #F6DF78            | Call to Actions (CTAs), botões e tipografia de impacto |

/\* Exemplo de implementação no style.css da HJE ou coin.css \*/

```
:root {
  --coin-neon-purple: #4D28FF;
  --coin-neon-blue: #4151FF;
  --coin-dark-blue: #1D27B4;
  --coin-base-blue: #2430DB;
  --coin-yellow-highlight: #F6DF78;
}

.coin-theme {
  background-color: var(--coin-dark-blue);
  color: white;
}

.coin-cta-button {
  background-color: var(--coin-yellow-highlight);
  color: var(--coin-dark-blue);
}
```

## 1.2. Tipografia e Fontes Específicas

As fontes oficiais escolhidas não são nativas da web e devem ser implementadas localmente usando a regra `@font-face`. O processo envolve baixar os arquivos (nos formatos .ttf ou .woff2) e armazená-los no diretório `frontend/static/fonts/` do projeto Django.

- **Títulos (COGNITOIN):** Lotus Eater Sans
- **Textos Principais:** Sackers Gothic
- **Textos Secundários:** Balgin

```
/* Implementação no CSS */
@font-face {
font-family: 'Lotus Eater Sans';
src: url('../fonts/LotusEaterSans.woff2') format('woff2');
font-weight: normal;
font-style: normal;
}

@font-face {
font-family: 'Sackers Gothic';
src: url('../fonts/SackersGothic.woff2') format('woff2');
font-weight: normal;
font-style: normal;
}

@font-face {
font-family: 'Balgin';
src: url('../fonts/Balgin.woff2') format('woff2');
font-weight: normal;
font-style: normal;
}

.coin-title {
font-family: 'Lotus Eater Sans', sans-serif;
}

.coin-text {
font-family: 'Sackers Gothic', sans-serif;
}
```

## 1.3. Elementos Visuais e Decorativos (Canva)

Para aplicar os elementos visuais fluidos e dinâmicos (estilo do "vision board" fornecido):

1. No Canva, selecione as formas orgânicas, gradientes e texturas que simulam a fumaça digital e os contornos neon.
2. Exporte esses elementos individualmente como **PNG com fundo transparente**. Imagens pesadas de fundo inteiro devem ser exportadas como **WebP**.
3. Armazene-os em `frontend/static/imgs/coin/`.

4. No CSS, utilize a propriedade `mix-blend-mode: screen`; ou `mix-blend-mode: overlay`; sobre os fundos azuis e roxos para criar a fusão perfeita do efeito tecnológico neon.

## 2. Estrutura do HTML: Derivação do home.html

O arquivo `core/templates/coin.html` deve estender ou importar seções estruturais do site da HJE para manter a coerência.

### 2.1. O que Manter Padrão (Reaproveitamento)

- **Navbar:** A barra de navegação do topo deve ser mantida usando a mesma diretiva (ex: `{% include 'navbar.html' %}` ou estendendo um `base.html`).
- **Footer:** Mantenha a estrutura de grid do rodapé original, preservando o e-mail oficial institucional de contato. Altere apenas os links de redes sociais para apontarem para as mídias exclusivas da CoIn.
- **Grid Dinâmico:** Analise as classes de flexbox ou CSS Grid da `home.html` que formatam os blocos de projetos. Reaproveite essas exatas classes (como os containers de "cards") para estruturar visualmente as seções de Palestrantes e Hackathons.

### 2.2. Construção e Especificidade do coin.html

A estrutura do novo arquivo deve isolar as customizações dentro de um container com a classe `coin-theme` para não interferir nas outras abas da HJE.

```
{% extends 'base.html' %}
{% load static %}

{% block content %}
<main class="coin-theme">
<!-- Hero Section Imersiva -->
<section class="coin-hero">
    
    <h1 class="coin-title">O FUTURO DA SAÚDE É COOPERATIVO E
INTELIGENTE</h1>
</section>

<!-- Resumo da Semana de IA -->
<section class="coin-about">...</section>

<!-- Palestrantes (Renderização Dinâmica via Backend) -->
<section class="coin-speakers">
    {% for dia, lista_palestrantes in dias_evento.items %}
        <h2 class="coin-title">Dia {{ dia }}</h2>
        <div class="grid-hje-padrao">
            {% for palestrante in lista_palestrantes %}
                <!-- Reuso do formato de card já padronizado -->
                <div class="card">...</div>
            {% endfor %}
        </div>
    {% endfor %}
</section>
</main>
{% endblock %}
```

```

        {% endfor %}
    </div>
    {% endfor %}
</section>

<!-- Patrocinadores e Hackathons -->
<section class="coin-sponsors">
    <h2>Apoio e Hackathons</h2>
    <a href="mailto:einstein.coding@outlook.com"
class="coin-cta-button">Seja um Patrocinador</a>
</section>
</main>
{% endblock %}

```

### 3. Programação do Modelo de Acesso e Banco de Dados (Django)

Como padrão da HJE, o site deve ser administrável e dinâmico. A gestão de cronograma, palestrantes e patrocinadores deve ser consumida do banco de dados, permitindo edições rápidas sem necessitar deploy de código HTML.

#### 3.1. Modelos (models.py)

Configure os modelos que representem a entidade Palestrante vinculada a um Dia específico da semana do evento.

```

from django.db import models

class PalestranteCoin(models.Model):
    nome = models.CharField(max_length=150)
    cargo = models.CharField(max_length=150)
    dia_apresentacao = models.IntegerField(help_text="Dia 1, 2, etc.")
    foto = models.ImageField(upload_to='coin/palestrantes/')
    ordem = models.IntegerField(default=0)

    def __str__(self):
        return f"{self.nome} - Dia {self.dia_apresentacao}"

```

#### 3.2. Lógica da View (views.py)

A rota do back-end deve consultar os dados, ordená-los e agrupá-los por dia para a injeção no template.

```

from django.shortcuts import render
from .models import PalestranteCoin

def coin_view(request):

```

```

palestrantes =
PalestranteCoin.objects.all().order_by('dia_apresentacao', 'ordem')
dias_evento = {}

for p in palestrantes:
    if p.dia_apresentacao not in dias_evento:
        dias_evento[p.dia_apresentacao] = []
    dias_evento[p.dia_apresentacao].append(p)

context = {
    'dias_evento': dias_evento,
    'tema_oficial': 'O FUTURO DA SAÚDE É COOPERATIVO E INTELIGENTE'
}
return render(request, 'coin.html', context)

```

### 3.3. Roteamento (urls.py)

```

from django.urls import path
from . import views

urlpatterns = [
    # ... outras rotas da hje ...
    path('coin/', views.coin_view, name='coin_page'),
]

```

**Nota final:** Na visualização home.html da HJE (página inicial principal), adicione um card estático no componente de "Eventos e Notícias" apontando a URL de link para {% url 'coin\_page' %}, conectando a raiz do site à nova extensão focada em IA.